Analysis Principles

1. The information domain of a problem must be represented and understood.
2. The **function** that the software is to be perform must be defined.
3. The behavior of the software must be represented
4. The model that depict information, function, and behavior must be partitioned in hierarchical fashion
5. The analysis process should move from essential information toward implementation details

Analysis Principles

- By applying these principles, the analysis principles, the analyst approaches a problem systematically .
- Information domain is examined so that the function may be understood more completely
- Models are used so that the characteristics of function and behavior can be communicated in a compact fashion.
- **Partitioning** is applied to reduce complexity

Analysis Principles

- In addition to these operational analysis, Davis suggests a set of guiding principles for requirement engineering
  - ***Understand the problem before you begin to create the analysis model. --***There is a tendency to rush to a solution, even before the problem is understood. This often leads to elegant software that solves the wrong problem!
  - ***Develop the prototypes that enable a user to understand how human/machine interaction will occur --*** Since the perception of the quality of software is often based on the perception of the "friendliness" of the interface, prototyping (and the iteration that results) are highly recommended

Analysis Principles

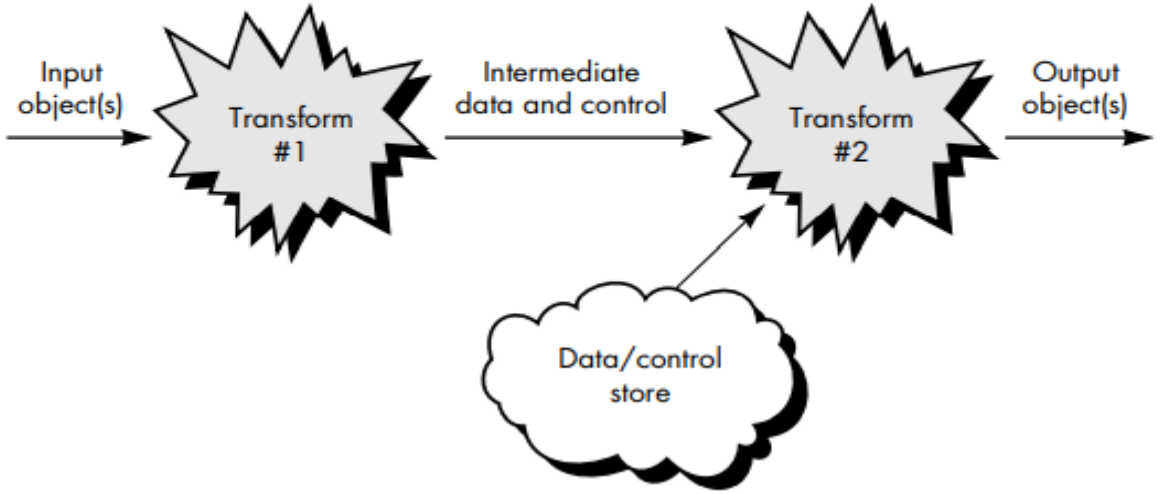
- **Record the origin of and the reason for every requirement--** This is the first step in establishing traceability back to the customer
- **Use multiple views of the requirement --**Building data, functional, and behavioral models provide the software engineer with three different views. This reduces the likelihood that something will be missed and increases the likely hood that inconsistency will be recognized
- **Rank the requirements --** Tight deadlines may preclude the implementation of every software requirement. If an incremental process model (Chapter 2) is applied, those requirements to be delivered in the first increment must be identified.
- **Work to eliminate ambiguity** -- Because most requirements are described in a natural language, the opportunity for ambiguity abounds. The use of formal technical reviews is one way to uncover and eliminate ambiguity.

Information Domain

- Software is **built** to accept the input, manipulate it on some way, and produce output.
- Software also **process** the event.
- An event represents some aspect of system control and is really nothing more than Boolean data – either on or off
- The information domain consists of three different views
- **Information content or data model** ◦
- shows the relationships among the data and control objects that make up the system
- **Information flow** ◦
- represents manner in which data and control objects change as each moves through system
- **Information structure** ◦
- representations of the internal organizations of various data and control items

Information Domain

FIGURE 11.3
Information flow and transformation



Modelling

- We create **functional models** to gain a better understanding of the actual entity to be built.
- When the entity is a physical thing (a building, a plane, a machine), we can build a model that is identical in form and shape but smaller in scale.
- However, when the entity to be built is software, our model must take a different form.
- It must be capable of representing the information that software transforms, the functions (and sub functions) that enable the transformation to occur, and the behavior of the system as the transformation is taking place.
- The second and third operational analysis principles require that we build models of function and behavior.

Functional models.

- Software transforms information, and in order to accomplish this, it must perform at least three generic functions: input, processing, and output.
- When functional models of an application are created, the software engineer focuses on problem specific functions.
- The functional model begins with a single context level model (i.e., the name of the software to be built). Over a series of iterations, more and more functional detail is provided, until a thorough delineation of all system functionality is represented.

Behavioral models

- Most software responds to events from the outside world.
- This stimulus/response characteristic forms the basis of the behavioral model.
- A computer program always exists in some state—an externally observable mode of behavior (e.g., waiting, computing, printing, polling) that is changed only when some event occurs

Importance of models

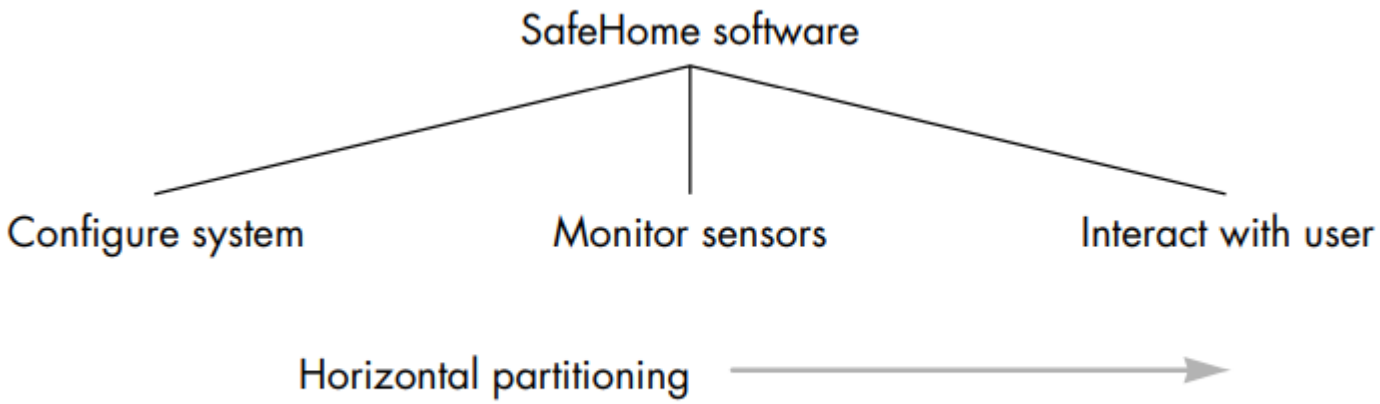
- Models created during requirements analysis serve a number of important roles:

- The model aids the analyst in understanding the **information, function, and behavior of a system**.
- The model becomes the **focal point for review**
- The model becomes the **foundation for design**

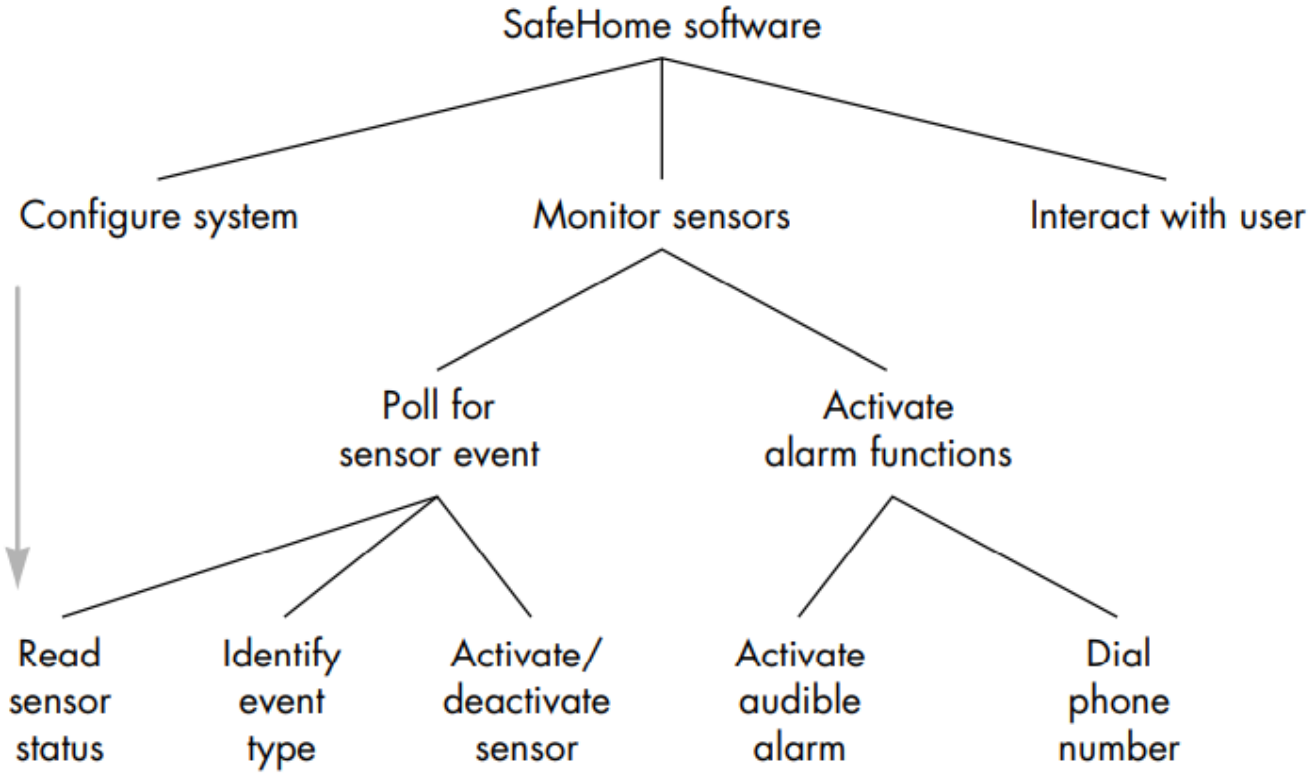
Partitioning

- Problems are often too large and complex to be understood as a whole.
- That's why we need to partition (divide) such problems into parts that can be easily understood and establish interfaces between the parts so that overall function can be accomplished.
- Analysis principle suggests that the information, functional, and behavioral domains of software can be partitioned.
- partitioning decomposes a problem into its constituent parts.
- Conceptually, we establish a hierarchical representation of function or information and then partition the uppermost element by:
- (1) exposing increasing detail by moving vertically in the hierarchy or
- (2) functionally decomposing the problem by moving horizontally in the hierarchy

Partitioning



Partitioning



Essential and Implementation Views

- An **essential view** of software requirements presents the functions to be accomplished and information to be processed without regard to implementation details.
- **Implementation view** of software requirements presents the real world manifestation of processing functions and information structures.
- software requirements engineering should focus on what the software is to accomplish, rather than on how processing will be implemented.
- Implementation view should not necessarily be interpreted as a representation of how.
- Rather, an implementation model represents the current mode of operation

Selecting the Prototyping Approach

- Analysis should be conducted regardless of the software engineering paradigm that is applied.
- However, the form that analysis takes will vary. I
- n some cases it is possible to apply operational analysis principles and derive a model of software from which a design can be developed.
- In other situations, requirements elicitation (via FAST, QFD, use-cases, or other "brainstorming" techniques is conducted, analysis principles are applied, and a model of the software to be built, called a prototype, is constructed for customer and developer assessment.
- Finally, some circumstances require the construction of a prototype at the beginning of analysis, since the model is the only means through which requirements can be effectively derived.
- The model then evolves into production software.

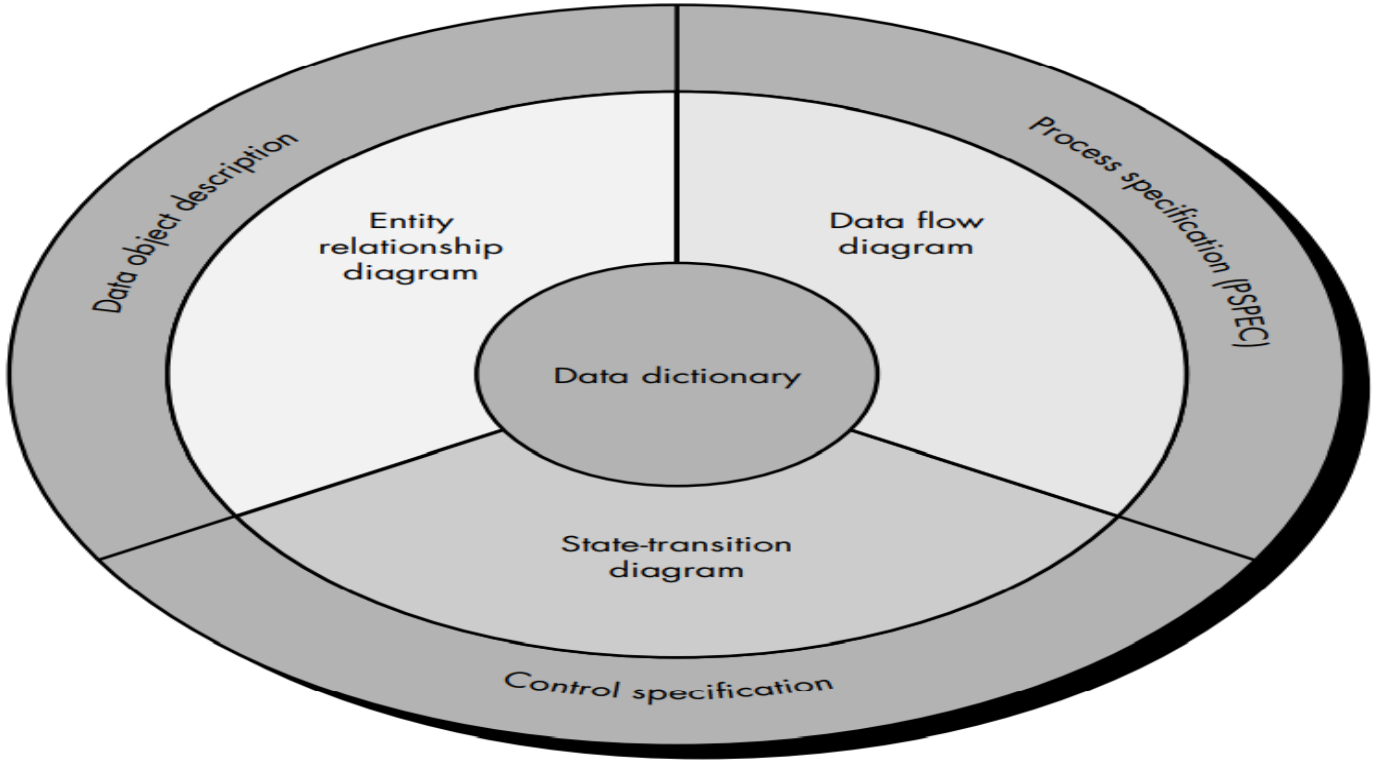
Selecting the prototype model

- The prototyping paradigm can be either **close-ended** or **open-ended**.
- The close-ended approach is often called t**hrowaway** prototyping. Using this approach, a prototype serves solely as a rough demonstration of requirements.
- It is then discarded, and the software is engineered using a different paradigm.
- An **open-ended** approach, called **evolutionary prototyping**, uses the prototype as the first part of an analysis activity that will be continued into design and construction.
- The prototype of the software is the first evolution of the finished system

Objectives of Analysis Modelling:

- It must establish **a basis** for the creation of software design.
- It must **describe** requirements of customer.
- It must define **set of requirements** which can be validated, once the software is built.

Elements of Analysis Model



Data Dictionary:

- A **repository** that contains description of all the data objects consumed and produced by the software.
- Three different diagrams surrounds the core.
- The **ERD** depicts the relationships between data objects
- The **ERD** is the notation that is used to conduct the data modelling activity.
- The attribute of each data object in the ERD can be described using **data object description** .

State Transition Diagram

- The state transition indicates how the **system behaves** as a consequences of external events.
- STD represents the various modes of behavior called the state.
- Information about the control aspects of the software is contained in the **control specification**

Data Flow Diagram

DFD provides the two purpose

1. To provide the an indication of how data are transformed as they move through the system.
2. To depict the function and sub functions that transform the data flow.

A description of each function presented in the DFD is contained in a **process Specification (PSPEC)**

Data Modelling

- Data modeling answers a set of specific questions that are relevant to any data processing application.
- What are the **primary data** objects to be processed by the system?
- What is the **composition of each data object** and what **attributes describe** the object?
- Where do the **the objects** currently reside?
- What are the **relationships** between each **object and other objects**?

Data Modelling

- Data modeling methods make use of the **entity relationship diagram**.
- The ERD, enables a software engineer to identify **data objects** and their relationships using a graphical notation.
- In the context of **structured analysis**, the ERD defines all data that are entered, stored, transformed, and produced within an application
- The ERD is especially useful for applications in which data and the relationships that govern data are complex

Data Objects

- A **data object** can be an :
- **external entity** (e.g., anything that produces or consumes information),
- **a thing** (e.g., a report or a display),
- **an occurrence** (e.g., a telephone call) or event (e.g., an alarm),
- **a role** (e.g., salesperson),
- **an organizational unit** (e.g., accounting department),
- **a place** (e.g., a warehouse),
- **or a structure** (e.g., a file).

Attribute

- Attributes define the properties of a data object and take on one of three different characteristics. They can be used to
- **name an instance of the data object,**
- **describe the instance, or**
- **make reference to another instance in another table.**

Attribute

Instance

Naming attributes

Ties one data object to another, in this case, owner

Identifier

Descriptive attributes

Referential attributes

Make	Model	ID#	Body type	Color	Owner
Lexus	LS400	AB123. . .	Sedan	White	RSP
Chevy	Corvette	X456. . .	Sports	Red	CCD
BMW	750iL	XZ765. . .	Coupe	White	LJL
Ford	Taurus	Q12A45. . .	Sedan	Blue	BLF

Cardinality and Modality

- Cardinality is the specification of the number of occurrences of one **object** that can be related to the number of occurrences of another **object.**
- Cardinality is usually expressed as simply **'one' or 'many**.
- For example, a husband can have only one wife , while a parent can have many children

Cardinality and Modality

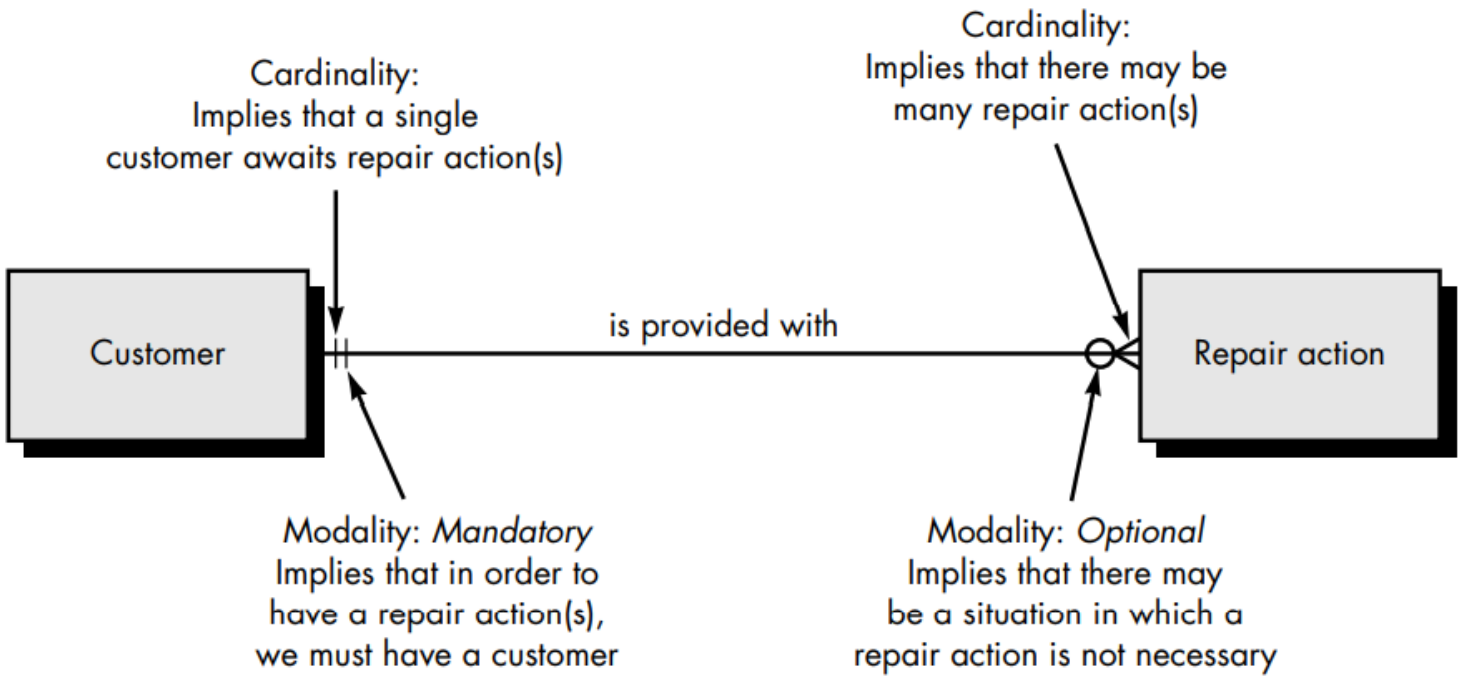
- **One-to-one (I:I)—** a student can have only one Roll number
- **One-to-many (I:N)—** a mother can have many children, but a child can have only one mother.
- **Many-to-many (M:N)—** an uncle can have many nephews, while a nephew can have many uncles

- Cardinality defines “the maximum number of objects that can participate in a relationship”.
- It does not, however, provide an indication of whether or not a particular data object must participate in the relationship.
- To specify this information, the data model adds modality to the object/relationship pair.

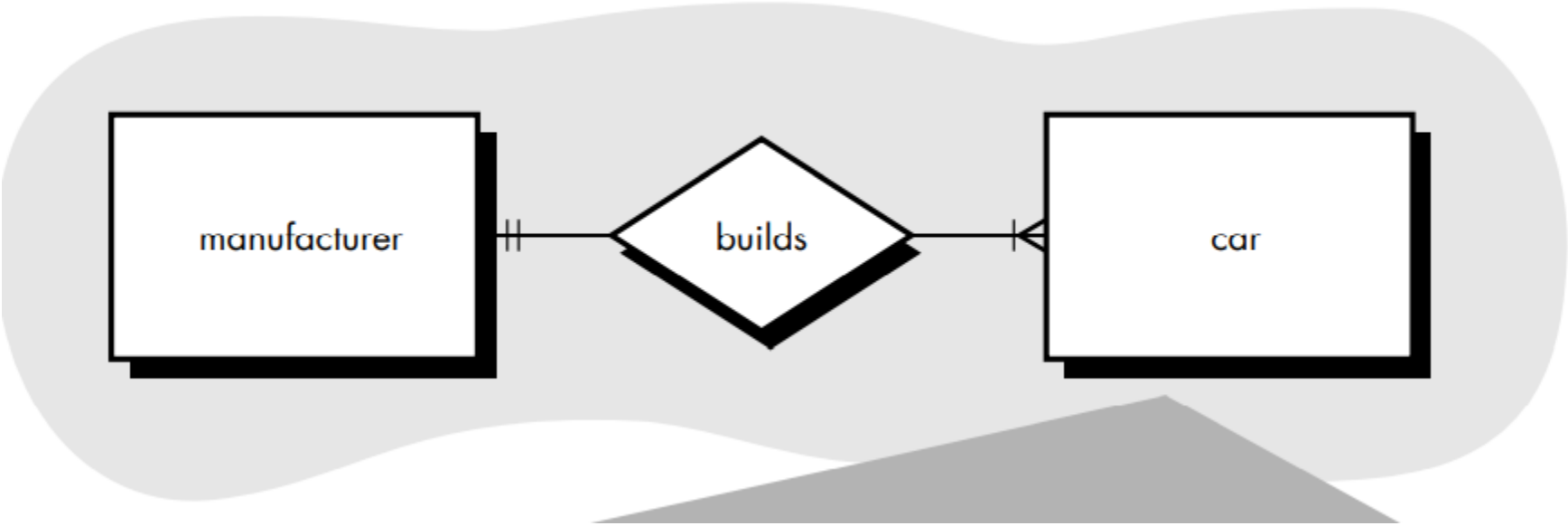
Modality

- The modality of a relationship is 0 if there is no explicit need for the relationship to occur or the relationship is optional.
- The modality is 1 if an occurrence of the relationship is mandatory.
- *Example : A customer indicates that there is a problem. If the problem is diagnosed as relatively simple, a single repair action occurs. However, if the problem is complex, multiple repair actions may be required*

Modality


Cardinality:
Implies that a single
customer awaits repair action(s)
Cardinality:
Implies that there may be
many repair action(s)
Customer
is provided with
Repair action
Modality: *Mandatory*
Implies that in order to
have a repair action(s),
we must have a customer
Modality: *Optional*
Implies that there may
be a situation in which a
repair action is not necessary

ER Diagram



ER Diagram

- Above figure shows the One to many relationship
- That is, a single customer can be provided with zero or many repair actions.
- The symbols on the relationship connection closest to the data object rectangles indicate cardinality.
- The vertical bar indicates one and the three-pronged fork indicates many.
- Modality is indicated by the symbols that are further away from the data object rectangles.
- The second vertical bar on the left indicates that there must be a customer for a repair action to occur.
- The circle on the right indicates that there may be no repair action required for the type of problem reported by the customer.

ER Diagram Notations

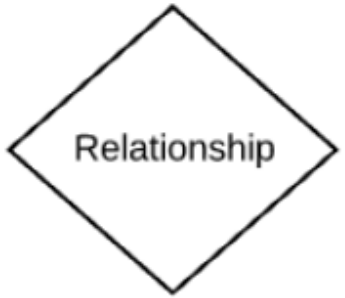
Attribute	Attribute	Attributes are characteristics of an entity, a many-to-many relationship, or a one-to-one relationship.
Multivalued Attribute	Multivalued attribute	Multivalued attributes are those that are can take on more than one value.
Derived Attribute	Derived attribute	Derived attributes are attributes whose value can be calculated from related attribute values.
Relationship	Relationship	Relationships are associations between or among entities.

Activate Windows
Go to Settings to activate W

ER Diagram Notation

Entity	Strong entity	These shapes are independent from other entities, and are often called parent entities, since they will often have weak entities that depend on them. They will also have a primary key, distinguishing each occurrence of the entity.
Weak Entity	Weak entity	Weak entities depend on some other entity type. They don't have primary keys, and have no meaning in the diagram without their parent entity.

ER Diagram Notation



Relationship

Relationships are associations between or among entities.

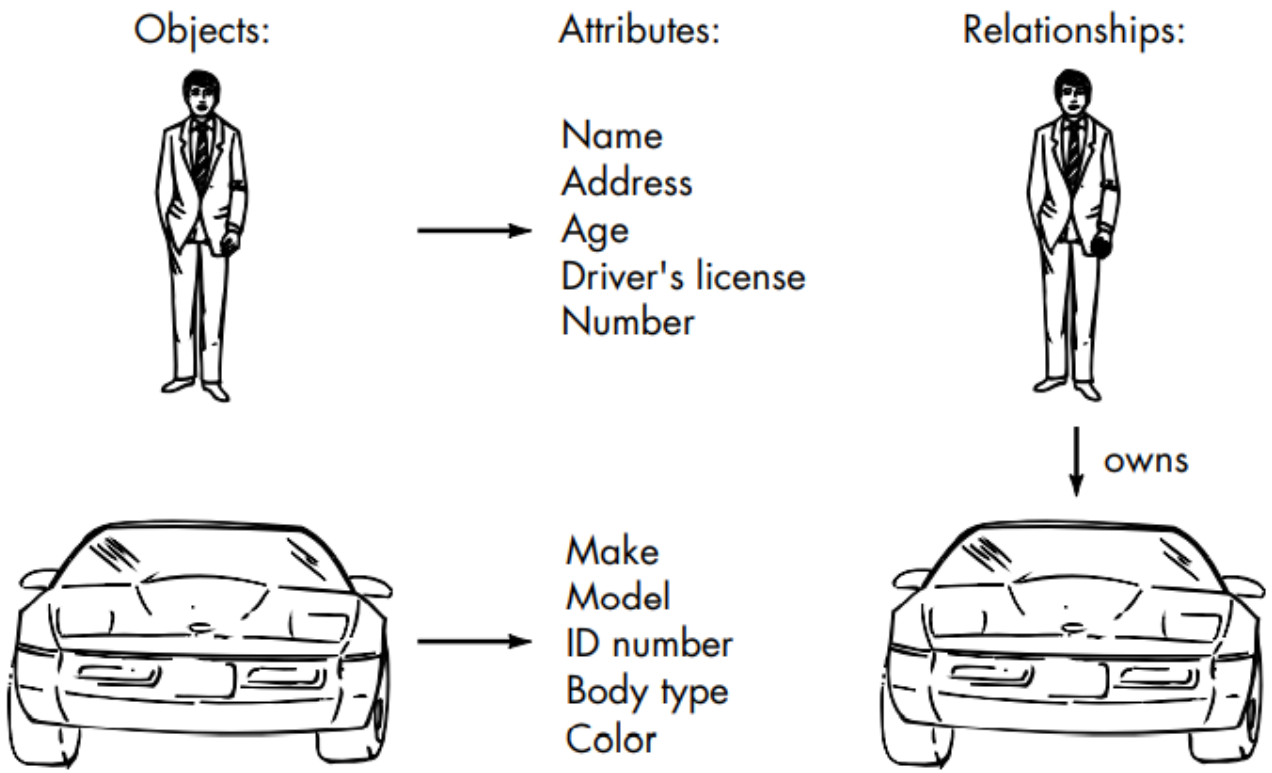
Weak relationship

Weak Relationships are connections between a weak entity and its owner.

Data Modelling

- The data model consists of three interrelated pieces of information: the **data object**, the **attributes that describe the data object**, and the **relationships** that connect data objects to one another

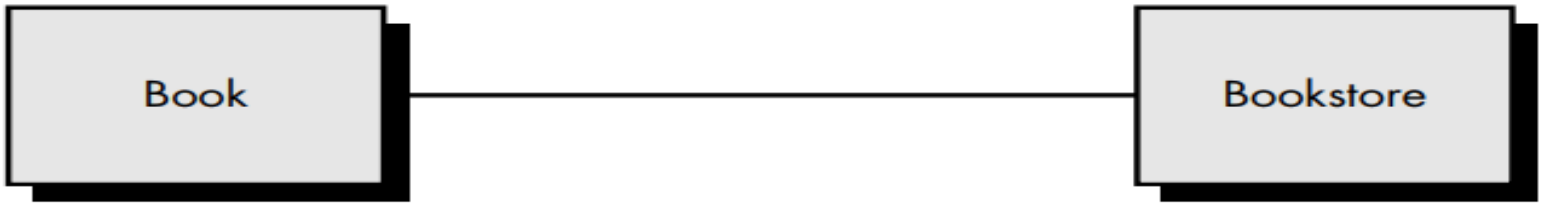
Data Modelling



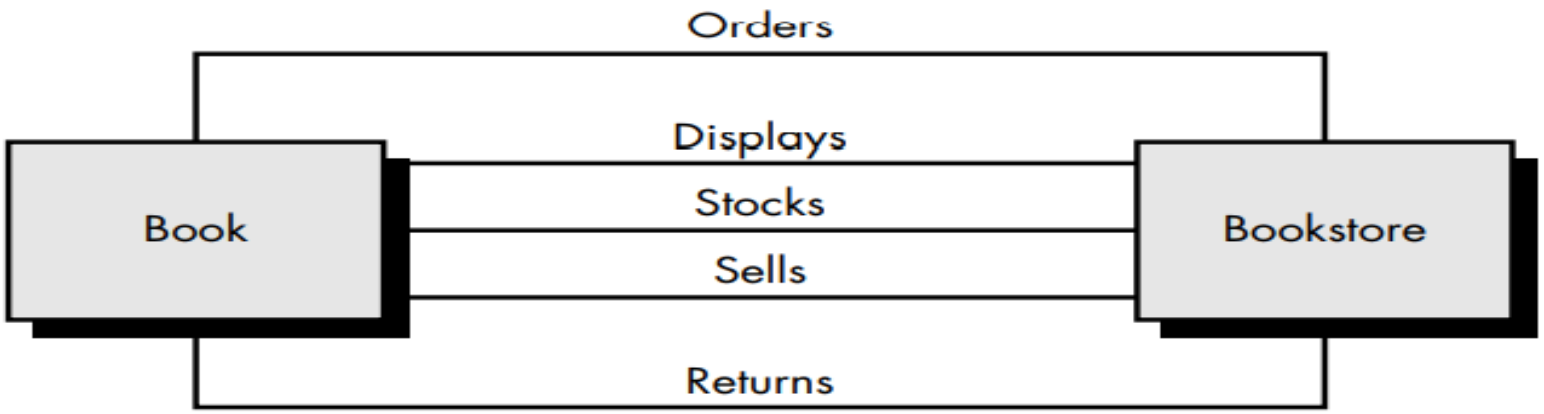
Relationships

- Relationships indicate the manner in which data objects are "connected" to one another.
- A bookstore **orders** books.
- A bookstore **displays** books.
- A bookstore **stocks** books.
- A bookstore **sells** books.
- A bookstore **returns** books.

Relationships



(a) A basic connection between objects



(b) Relationships between objects

Relationship

- It is important to note that object/relationship pairs are bidirectional. That is, they can be read in either direction.
- A bookstore **orders** books or books are **ordered** by a bookstore.

Cardinality and Modality

- The elements of data modeling—data objects, attributes, and relationships— provide the basis for understanding the information domain of a problem.
- However, additional information related to these basic elements must also be understood.
- We have defined a set of objects and represented the object/relationship pairs that bind them.
- But a simple pair that states: **object X** relates to **object Y** does not provide enough information for software engineering purposes.
- We must understand how many occurrences of **object X** are related to how many occurrences of **object Y**.
- This leads to a data modeling concept called **cardinality**

Cardinality and Modality

- The elements of data modeling—data objects, attributes, and relationships— provide the basis for understanding the information domain of a problem.
- However, additional information related to these basic elements must also be understood.
- We have defined a set of objects and represented the object/relationship pairs that bind them.
- But a simple pair that states: **object X** relates to **object Y** does not provide enough information for software engineering purposes.
- We must understand how many occurrences of **object X** are related to how many occurrences of **object Y**.
- This leads to a data modeling concept called **cardinality**

Modeling

We create functional models to gain a **better understanding** of the actual entity to be built.

- **Data model**
- shows relationships among system objects
- **Functional model**
- software converts information and to accomplish this, it must perform at least three common tasks- input, processing and output.
- When functional models of an application are created, the software engineer emphasizes problem specific tasks.
- The functional model begins with a single reference level model (i.e., name of the software to be built). I
- n a series of iterations, more and more functional detail is given, until all system functionality is fully represented.

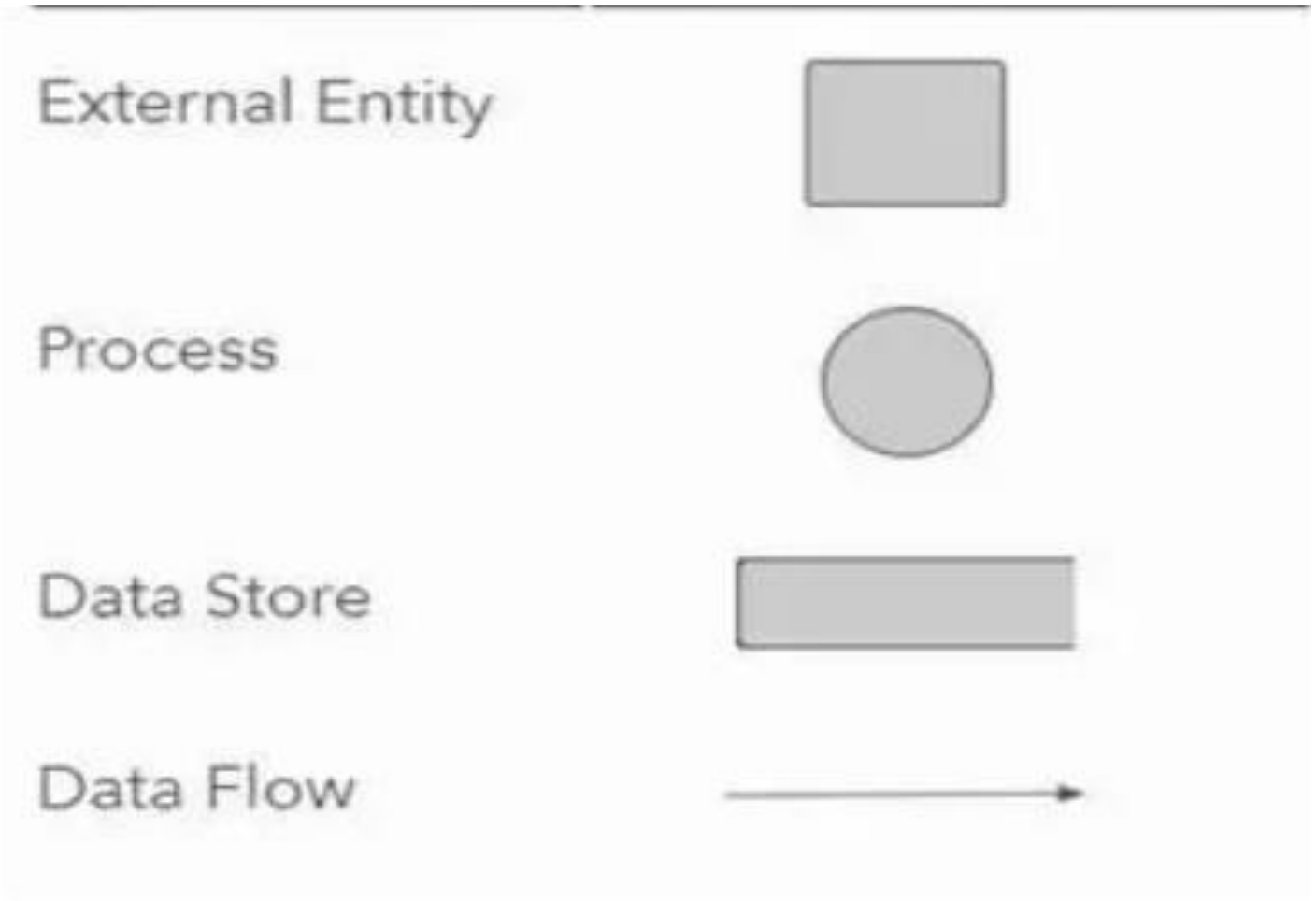
Functional Model

- The DFD may be used to represent a system or software at any level of abstractions.
- DFDS may be partitioned into levels that represents increasing information flow and functional details
- Therefore, the DFD provides a mechanism for functional modelling as as information flow modelling
- A level 0 DFD, also called a fundamental model or a context model, represents the entire software element as a single bubble with input and output data indicated by incoming and outing arrows, respectively

Representation of Data Flow

- External Entity is :An external entity can represent a human, system or subsystem. It is where certain data comes from or goes to system Represented by Rectangle
- Process : A process is a business activity or function where the manipulation and transformation of data take place. Represented by circle
- Data Store :A data store represents the storage of persistent data required and/or produced by the process.
- Data Flow :A data flow represents the flow of information, with its direction represented by an arrowhead that shows at the end(s) of flow connector.

Symbols used in Data Flow Diagram



Levels of DFD Diagrams

1.Level 0 DFD

Also known as Context Diagram or **Functional System Model :**

Represents the entire software elements as a **single bubble** with input and output data indicated by incoming and outgoing arrows.

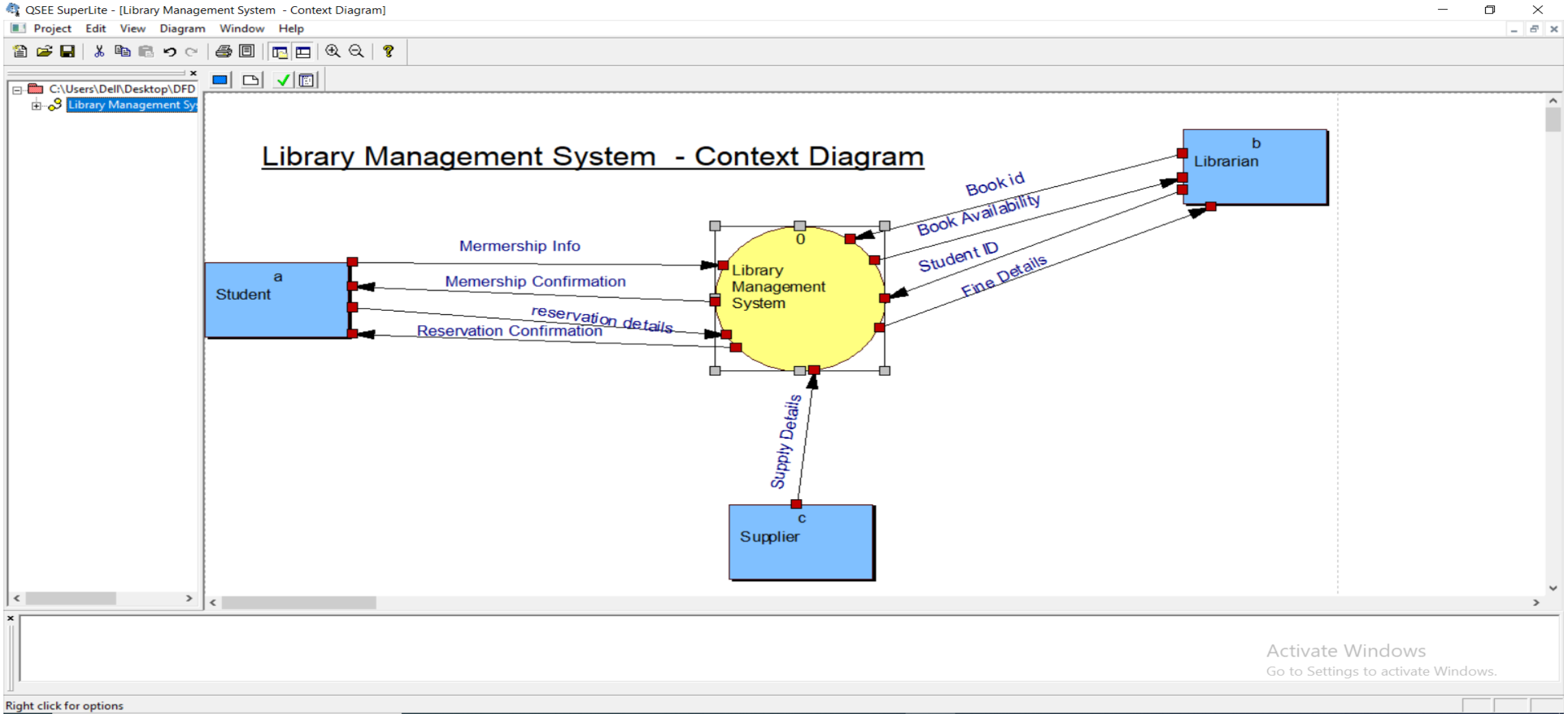
2. Level 1 DFD :

- are still a general overview, but they go into more detail than a context diagram.
- In a level 1 data flow diagram, the single process node from the context diagram is broken down into sub processes.
- As these processes are added, the diagram will need additional data flows and data stores to link them together.

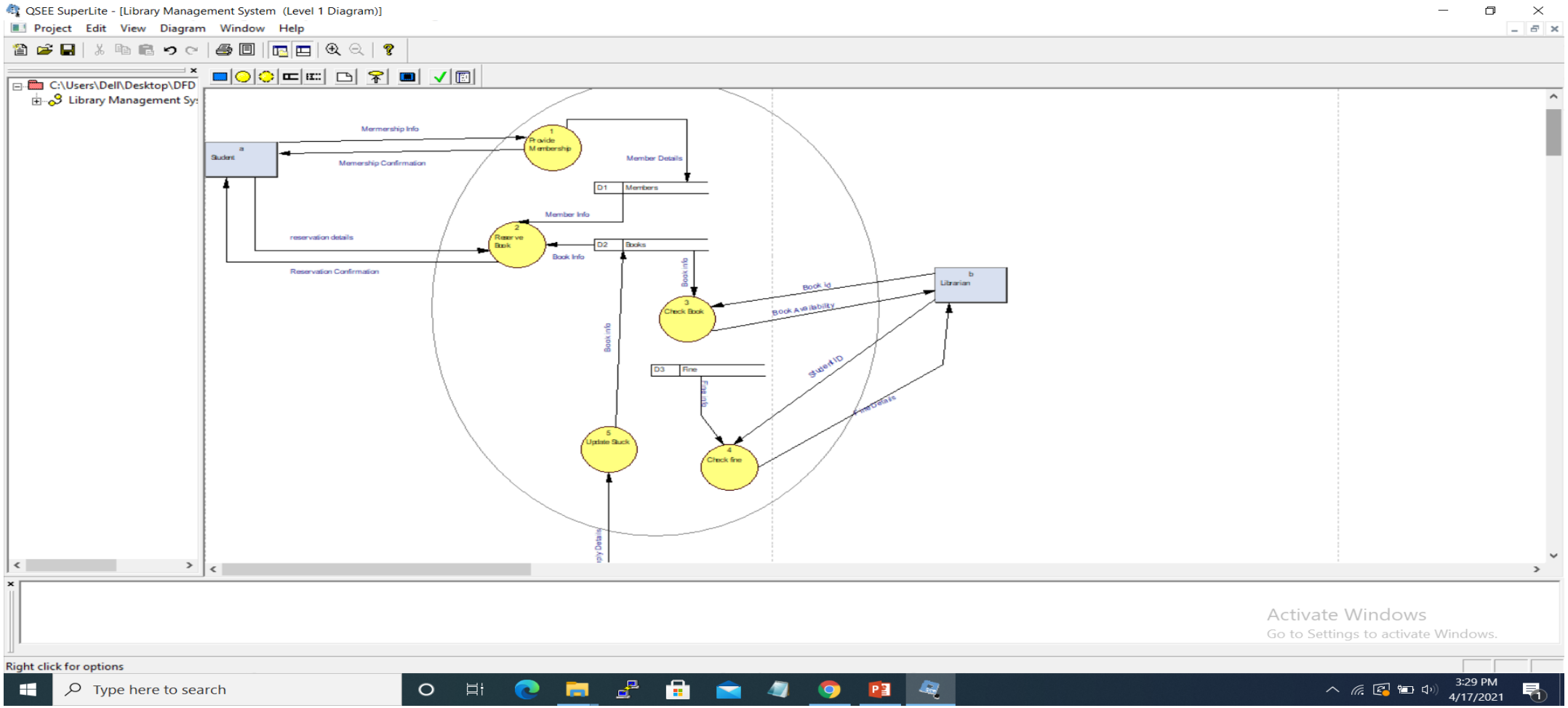
3. Level 2 DFD

- simply break processes down into more detailed sub processes.

DFD 0 or Context Diagram



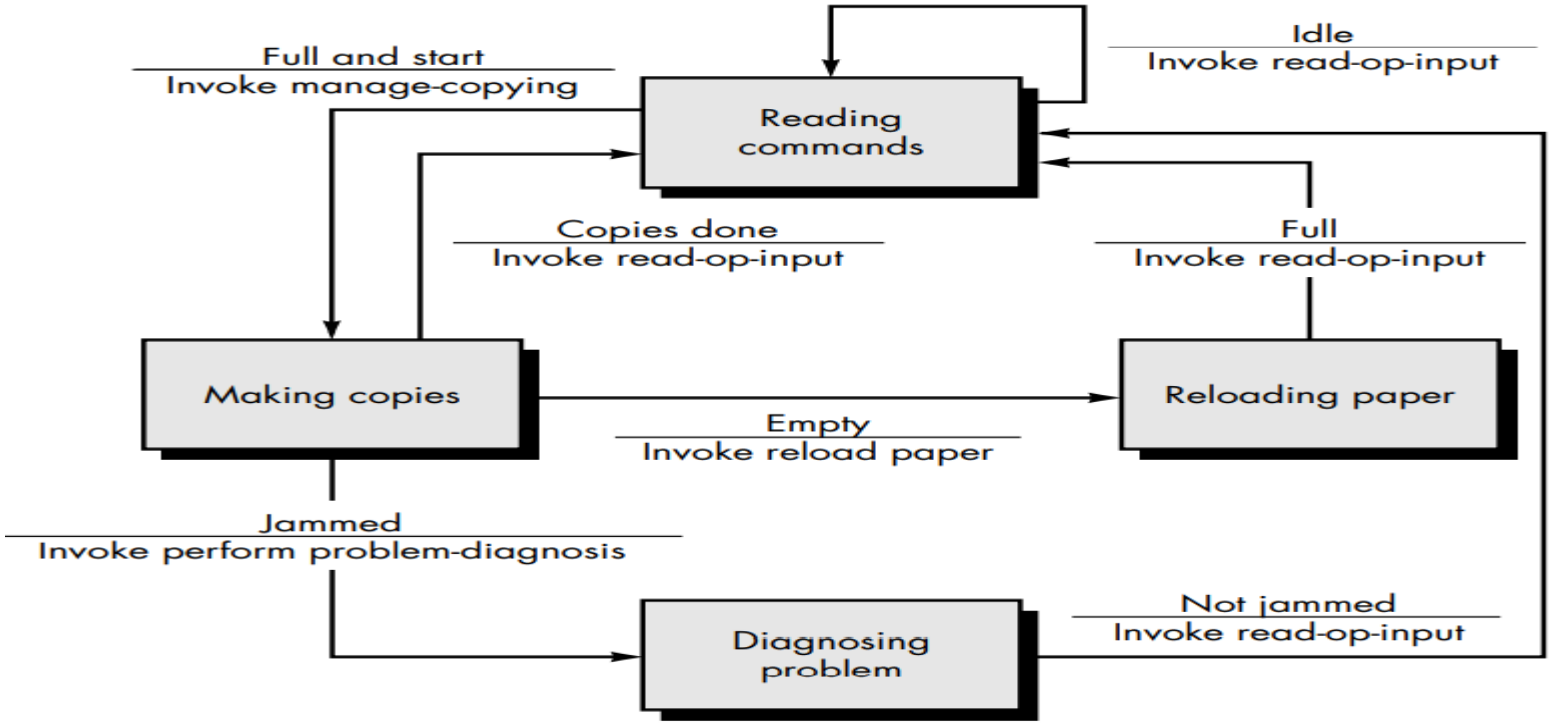
DFD level 1


QSEE SuperLite - [Library Management System (Level 1 Diagram)]
Project Edit View Diagram Window Help
C:\Users\Dell\Desktop\DFD
Library Management Sy
Mermership Info
Mermership Confirmation
Student
1 Provide Membership
Member Details
D1 Members
Member Info
reservation details
2 Reserve Book
D2 Books
Book Info
Reservation Confirmation
Book info
3 Check Book
Book Id
Book Availability
Librarian
Book info
D3 Fine
Fine info
Student ID
Fine details
5 Update Stuck
4 Check fine
Right click for options
Activate Windows
Go to Settings to activate Windows.
Type here to search
3:29 PM
4/17/2021

Behavioral Modelling

- **state transition diagram** are used to represent the behavior of the system to various external conditions and inputs.
- It graphically represents how a system in one state switches to another state on the action of external stimuli and control signals.
- A **state** is any observable mode of behavior.

Behavioral Modelling



Behavior Modelling

- System states are represented by **a rectangular shape by rounded corner** and arrows are used to represent transitions between states.
- Each arrow is labelled as a ruled expression A/B.
- Top value represents the event responsible for the transition and bottom value represents the action that occurs as a consequent of the event during transition.

Behavioral Modelling

- Fig shows a state transition diagram for photocopier software.
- When the paper tray is full and the start button is pressed, the system moves from the reading commands state to the making copies state.
- Photocopies software will make photocopies only when it not in jammed state and exists in start state.
- If it is in jammed state, then the problem is diagnosed and removed.
- If photocopier is empty means do not have blank papers then the papers are reloaded.

Data Dictionary

- The data dictionary is an organized listing of all data elements that are pertinent to the system.
- Data are organized with precise, rigorous definitions
- Both user and system analyst will have a common understanding of inputs, outputs, components of stores and even intermediate calculations

Data Dictionary

- **Name**—the primary name of the data or control item
- **Alias**—other names used for the first entry.
- **Where-used/how-used**—a listing of the process and how it is used
- **Content description**—a notation for representing content.
- **Supplementary information**—other information about data types, preset values, restrictions or limitations, and so forth.

Data Dictionary

- The notation used to develop a content description is noted in the following table:

Data Construct	Notation	Meaning
	=	is composed of
Sequence	+	and
Selection	[|]	either-or
Repetition	$\{\ \}^n$	*n* repetitions of
	( )	optional data
	...	delimits comments

Example of Data Dictionary

- **name**: telephone number
- **aliases**: none
- **where used/how used**: dial phone (input)
- **description**: telephone number = [local number / long distance number]
- **local number** = prefix + access number
- **long distance number** = 1 + area code + local number
- **area code** = [800 | 888 | 561]
- **prefix** = *a three digit number that never starts with 0 or 1*
- **access number** = * any four number string *